

# Text Preprocessing and TF-IDF



```
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder
import re
import os

# Download NLTK resources

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Download the missing 'punkt\_tab' data package

nltk.download('punkt\_tab') # This line is added to download the necessary data

# Sample data

data = {
'text': \[
"The quick brown fox jumps over the lazy dog.",
"I love programming in Python!",
"Natural Language Processing is fascinating.",
"Data Science is an interdisciplinary field.",
"Machine learning is a subset of artificial intelligence."
],
'label': \['animal', 'programming', 'NLP', 'data science', 'machine learning']
}

# Create a DataFrame

df = pd.DataFrame(data)

# Text cleaning function

def clean\_text(text):
\# Remove special characters and digits
text = re.sub(r'\[^a-zA-Z\s]', '', text)
\# Convert to lowercase
text = text.lower()
```

```

return text

# Lemmatization function

def lemmatize\_text(text):
    lemmatizer = WordNetLemmatizer()
    words = nltk.word\_tokenize(text)
    lemmatized\_text = ' '.join([lemmatizer.lemmatize(word) for word in words])
    return lemmatized\_text

# Remove stop words function

def remove\_stop\_words(text):
    stop\_words = set(stopwords.words('english'))
    words = text.split()
    filtered\_text = ' '.join([word for word in words if word not in stop\_words])
    return filtered\_text

# Apply text cleaning, lemmatization, and stop word removal

df['cleaned\_text'] = df['text'].apply(clean\_text)
df['lemmatized\_text'] = df['cleaned\_text'].apply(lemmatize\_text)
df['final\_text'] = df['lemmatized\_text'].apply(remove\_stop\_words)

# Label encoding

label\_encoder = LabelEncoder()
df['encoded\_label'] = label\_encoder.fit\_transform(df['label'])

# TF-IDF representation

tfidf\_vectorizer = TfidfVectorizer()
tfidf\_matrix = tfidf\_vectorizer.fit\_transform(df['final\_text'])

# Convert TF-IDF matrix to DataFrame

tfidf\_df = pd.DataFrame(tfidf\_matrix.toarray(),
    columns=tfidf\_vectorizer.get\_feature\_names\_out())

# Save outputs

output\_dir = 'output'
os.makedirs(output\_dir, exist\_ok=True)

df.to\_csv(os.path.join(output\_dir, 'cleaned\_data.csv'), index=False)
tfidf\_df.to\_csv(os.path.join(output\_dir, 'tfidf\_representation.csv'), index=False)

print("Outputs saved successfully.") Explain the code

```

This script performs **text preprocessing** and **feature extraction** using **TF-IDF** for a small dataset. It includes text cleaning, tokenization, lemmatization, stop word removal, label encoding, and conversion to TF-IDF vectors. Here's a breakdown:

## Imports and Setup

```
python

import pandas as pd, numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder
import re, os
```

- These libraries are used for data handling (`pandas`, `numpy`), text processing (`nltk`), machine learning preprocessing (`scikit-learn`), and system operations (`os`, `re`).

## NLTK Downloads

```
python

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tab') # ❌ Mistake here: This resource doesn't exist
```

- Downloads necessary NLP resources like:
  - `punkt`: for tokenization
  - `stopwords`: list of common stop words
  - `wordnet`: for lemmatization
- ⚠️ `punkt\_tab` is **not** a valid resource — this line should be removed or corrected.

## Sample Dataset

```
python

data = {
    'text': [...],
    'label': [...]
}
df = pd.DataFrame(data)
```

- Creates a small sample dataset with 5 sentences and their labels (categories).

## Text Cleaning Function

python

```
def clean_text(text):
    text = re.sub(r'^a-zA-Z\s]', '', text) # Removes digits and special characters
    text = text.lower() # Converts to lowercase
    return text
```

## Lemmatization

python

```
def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    words = nltk.word_tokenize(text)
    return ' '.join([lemmatizer.lemmatize(word) for word in words])
```

- Converts words to their base form using WordNet (e.g., "jumps" → "jump").

## Stop Word Removal

python

```
def remove_stop_words(text):
    stop_words = set(stopwords.words('english'))
    words = text.split()
    return ' '.join([word for word in words if word not in stop_words])
```

- Removes commonly used English words that don't add much meaning (e.g., "is", "the").

## Apply All Preprocessing Steps

python

```
df['cleaned_text'] = df['text'].apply(clean_text)
df['lemmatized_text'] = df['cleaned_text'].apply(lemmatize_text)
df['final_text'] = df['lemmatized_text'].apply(remove_stop_words)
```

## Label Encoding

python

```
label_encoder = LabelEncoder()
df['encoded_label'] = label_encoder.fit_transform(df['label'])
```

- Converts string labels (like `programming`) to integers for machine learning.

## TF-IDF Vectorization

python

```
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(df['final_text'])
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf_vectorizer.get_feature_names_out())
```

- Converts text into numerical form using **Term Frequency-Inverse Document Frequency**, capturing word importance.

## Saving the Outputs

python

```
output_dir = 'output'
os.makedirs(output_dir, exist_ok=True)
df.to_csv(os.path.join(output_dir, 'cleaned_data.csv'), index=False)
tfidf_df.to_csv(os.path.join(output_dir, 'tfidf_representation.csv'), index=False)
```

- Saves both the processed data and the TF-IDF features to CSV files.

## Final Output

python

```
print("Outputs saved successfully.")
```

Would you like help fixing the issues or modifying this script for a larger dataset?



write viva questions with answers of above code

Here are **viva questions with answers** based on the code you shared:

### ◆ 1. What is the purpose of this code?

**Answer:**

The purpose of this code is to preprocess text data, convert it into numerical features using TF-IDF vectorization, and save the cleaned data and vectorized output to CSV files for further use in machine learning or natural language processing tasks.

### ◆ 2. What are the main preprocessing steps applied to the text?

**Answer:**

The preprocessing steps include:

- Removing special characters and digits

- Converting text to lowercase
  - Tokenization
  - Lemmatization (reducing words to their base form)
  - Stop word removal (removing common words like "is", "the")
- 

### ◆ 3. What is the use of `WordNetLemmatizer`?

#### Answer:

`WordNetLemmatizer` from NLTK is used to reduce words to their base or dictionary form (lemma). For example, "running" becomes "run", helping to standardize words for better analysis.

---

### ◆ 4. Why is stop word removal important?

#### Answer:

Stop words are common words that usually do not carry significant meaning. Removing them reduces noise in the data and helps focus on meaningful words that contribute more to the understanding of the text.

---

### ◆ 5. What is TF-IDF and why is it used?

#### Answer:

TF-IDF (Term Frequency-Inverse Document Frequency) is a technique to convert text into numerical vectors by weighing words based on how often they appear in a document versus across all documents. It helps highlight important words while downplaying frequently occurring but less meaningful ones.

---

### ◆ 6. What does `LabelEncoder` do in this code?

#### Answer:

`LabelEncoder` converts categorical text labels (like "NLP", "data science") into numeric labels (like 0, 1, 2) which are required for machine learning models.

---

### ◆ 7. Why do we use `re.sub()` in the `clean_text` function?

#### Answer:

`re.sub()` is used to remove unwanted characters like punctuation, digits, and symbols by replacing them with an empty string, keeping only alphabets and whitespace.

---

### ◆ 8. What are the files saved at the end of the code?

#### Answer:

Two CSV files are saved:

1. `cleaned_data.csv`: Contains original text, cleaned text, lemmatized text, final text, and encoded labels.
2. `tfidf_representation.csv`: Contains the TF-IDF vector representation of the final cleaned text.

♦ **9. What is the mistake in the line ``nltk.download('punkt_tab')``?**

**Answer:**

``punkt_tab`` is not a valid NLTK resource. This line should be removed, as ``punkt`` (which is already downloaded) includes necessary tokenization data.

♦ **10. What would happen if we didn't lemmatize or remove stop words?**

**Answer:**

Not lemmatizing could result in multiple forms of the same word being treated separately, leading to data redundancy. Not removing stop words would introduce noise, reducing the quality of the feature representation.

Would you like more advanced questions, or questions targeted toward interviews or academic exams?



yes

Here are **advanced viva questions** with answers that go deeper into the concepts used in the code and its applications:

♦ **1. Explain the difference between Lemmatization and Stemming. Why is Lemmatization preferred in this code?**

**Answer:**

**Stemming** is a process where words are reduced to their root form by chopping off prefixes or suffixes (e.g., "running" becomes "run"). **Lemmatization**, on the other hand, considers the context of the word and returns the base form (e.g., "running" becomes "run" only if the word is a verb, otherwise it stays as "running").

**Lemmatization** is preferred in this code because it produces more accurate and meaningful base words, reducing words to their dictionary form, making the data more consistent for further processing.

♦ **2. How does TF-IDF handle the importance of terms across documents?**

**Answer:**

TF-IDF assigns higher weights to terms that appear frequently in a single document but are rare across all documents. It consists of two components:

- **Term Frequency (TF):** Measures how frequently a term occurs in a document.
- **Inverse Document Frequency (IDF):** Measures how rare a term is across all documents in the corpus.

The formula is:

$$TF - IDF(t, d) = TF(t, d) \times \log \left( \frac{N}{DF(t)} \right)$$

Where:

- $t$  is the term,
- $d$  is the document,
- $N$  is the total number of documents,
- $DF(t)$  is the number of documents containing the term  $t$ .

This method helps to down-weight common terms like "the" and emphasize more informative, rare terms.

### ♦ 3. How would you modify the code to handle a larger corpus of text?

**Answer:**

To handle a larger corpus, you would need to:

- **Optimize memory usage** by using batch processing for text cleaning, lemmatization, and stop word removal, especially for huge datasets.
- **Use sparse matrices** more efficiently since the TF-IDF representation is sparse, and handling it efficiently using `scipy.sparse` or similar tools would be necessary.
- **Parallel processing** can be employed to speed up text preprocessing (e.g., using Python's `multiprocessing` or libraries like `joblib`).
- **External storage** might be used to save intermediate results, such as using databases or distributed file systems like Hadoop or cloud-based storage.

### ♦ 4. What are the challenges of using the TF-IDF approach, and how can they be mitigated?

**Answer:**

Challenges of TF-IDF include:

- **Dimensionality:** The vector representation can be very sparse and large, especially with a large vocabulary. This can lead to **high memory consumption** and computational inefficiency.
  - **Mitigation:** Techniques like **Dimensionality Reduction** (e.g., PCA, LDA) or feature selection (removing low variance features) can reduce the number of features.
- **Inability to capture semantics:** TF-IDF treats each word as an independent feature, ignoring the context or semantics of the words.
  - **Mitigation:** Using more advanced models like **Word2Vec**, **GloVe**, or **transformers** (e.g., BERT) can capture semantic meaning and relationships between words.

### ♦ 5. What are the limitations of using `LabelEncoder` for encoding labels?

**Answer:**

`LabelEncoder` converts categorical labels into numerical labels, but it has some limitations:

- It **assumes an ordinal relationship** between the labels, even if they are nominal (without any inherent order).
  - **Mitigation:** For nominal data, **One-Hot Encoding** is preferred, which creates binary columns for each class.
- It is not suitable for multi-class problems when there is a **need for multi-label classification** (i.e., an instance can belong to more than one class).
  - **Mitigation:** Use multi-label encoders like **MultiLabelBinarizer**.

## ◆ 6. What would happen if the text contains multiple languages, and how could the code be adapted for multilingual data?

### Answer:

If the text contains multiple languages, certain functions (e.g., **stop word removal** or **lemmatization**) might fail, as they are language-specific. To handle multilingual data:

- **Language detection:** First, detect the language of each document (using libraries like `langdetect`).
- **Multilingual stop words and lemmatization:** Use multilingual stop word lists and lemmatizers, such as **spaCy**'s multilingual models, or use pre-trained models that handle multiple languages like **fastText**.
- **TF-IDF:** TF-IDF can still be applied across multilingual datasets, but it might require tokenization methods that support multiple languages.

## ◆ 7. How do you ensure the scalability of this code when deployed in a production environment for text classification?

### Answer:

To ensure scalability in a production environment:

- **Asynchronous processing:** Use task queues (e.g., Celery, Apache Kafka) to handle preprocessing tasks asynchronously, allowing parallel processing of large datasets.
- **Caching:** Cache intermediate results (e.g., TF-IDF vectors) to avoid redundant processing.
- **Model updates:** Implement a robust pipeline for model retraining to ensure that the system can handle new data and evolve over time.
- **Containerization:** Use Docker to package the application, which allows easy deployment and scaling across multiple servers in a cloud environment.

## ◆ 8. How would you evaluate the performance of the text preprocessing pipeline?

### Answer:

To evaluate the preprocessing pipeline:

- **Accuracy in cleaning:** Check if the cleaning process (removing special characters, stop words, etc.) retains meaningful information while removing unnecessary noise. Use **manual inspection** or

**automated metrics** (e.g., precision, recall on the cleaned text).

- **Impact on model performance:** After preprocessing, evaluate the model's performance (e.g., text classification accuracy) on a downstream task. A **cross-validation** approach can be used to assess how preprocessing affects model generalization.
- **TF-IDF matrix sparsity:** Measure the sparsity of the TF-IDF matrix to ensure efficient use of memory. Techniques like **sparse matrix operations** can help measure and reduce inefficiencies.

## ◆ 9. What alternatives to TF-IDF could be used for feature extraction in NLP tasks?

**Answer:**

Alternatives to TF-IDF include:

- **Word embeddings (Word2Vec, GloVe):** These models capture semantic meaning and relationships between words.
- **Contextual embeddings (BERT, GPT):** These transformer-based models generate word representations considering the context in which the word appears.
- **Bag of Words (BoW):** An earlier, simpler method where text is represented by a vector of word counts.
- **Latent Dirichlet Allocation (LDA):** A topic modeling approach where each document is represented as a distribution over topics, which can be used as features for classification.

These advanced questions cover broader aspects of text preprocessing, feature extraction, and scalability. Would you like to dive deeper into any specific area?

Do you like this personality?